

Trabalho Prático 1

Agentes conversacionais de busca

Vinicius Trindade Dias Abel
Matrícula: 2020007112

Universidade Federal de Minas Gerais (UFMG)
Belo Horizonte – MG – Brasil

viniciustda@ufmg.br

1. Introdução

Este projeto trata do problema de encontrar a rota mais curta entre universidades em Belo Horizonte, utilizando um algoritmo de busca sem informação (Dijkstra) ou um algoritmo de busca com informação (A^*), sendo que heurística utilizada avalia a distância euclidiana entre os nós. O objetivo é permitir que um agente conversacional receba uma solicitação do usuário, identifique a origem e o destino (entre universidades), e determine o melhor caminho a ser percorrido, tanto em termos de distância, descrevendo a rota encontrada, informando a distância total e gerando uma imagem e uma animação da rota.

O grafo utilizado foi construído a partir de dados reais obtidos via OpenStreetMap, com o auxílio da biblioteca OSMnx. Ele representa o sistema viário da cidade de Belo Horizonte - MG, filtrado para incluir apenas locais classificados como universidades.

- Número de nós: 2149
- Número de arestas: 3098 (no arquivo grafo.txt aparece o dobro, mas é para indicar que são não direcionadas)
- Número de destinos (universidades): 39 (As que aparecem com o mesmo nome são unidades diferentes. Entretanto não foi especificado de qual unidade se trata)
- Algoritmos implementados: Dijkstra e A^*

Esses algoritmos foram implementados em C++ por questões de desempenho e são integrados com a interface Python por meio de arquivos intermediários e um sistema de compilação automatizado.

2. Arquitetura do Sistema

A arquitetura da solução foi desenhada para integrar três componentes principais:

1- Interface Conversacional (Agente com SmolAgents + Gradio) - Arquivo `agent.py`:

- O agente implementado utiliza a LLM qwen3.
- Existem duas ferramentas (@tool) implementadas para o agente.
 - A primeira (`list_nodes_tool`) lista todas as universidades existentes no grafo.
 - A segunda (`search_route_tool`) encapsula funcionalidades. Ela busca a rota e a distância total, e gera a imagem e o GIF da rota.
- As ferramentas são criadas com base nas funções implementadas em `agent_tools.py`.
- `agent_tools.py` integra os componentes do projeto com funções que rodam os outros códigos implementados. Existem funções para criar o grafo, compilar c++ e executar os algoritmos de busca no c++ compilado. Além disso, tem funções para criar a imagem e o GIF da rota, e para criar uma descrição da rota com base no resultado gerado pelo executável.
- Com o Gradio, é criada uma interface para interação com o usuário. Permitindo que sejam feitas solicitações ao agente, receber as respostas e visualizar imagem e GIF gerados.

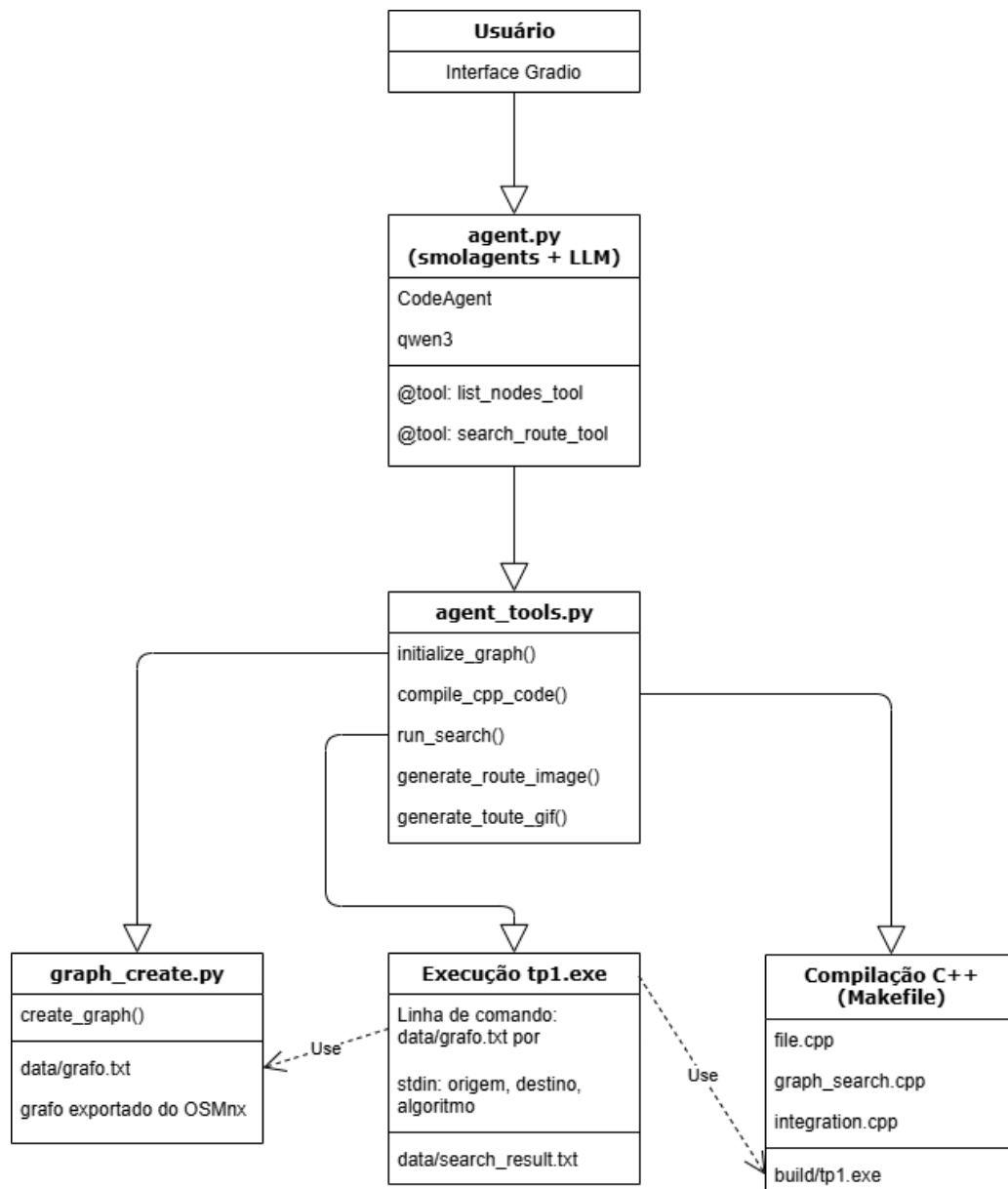
2- Construção e Manipulação do Grafo (Python com OSMnx) - Arquivo `graph_create.py`:

- O grafo é criado a partir de dados do OpenStreetMap usando a biblioteca OSMnx.
- Cada nó do grafo representa um ponto geográfico com coordenadas, nome e tipo.
- A representação do grafo é salva em arquivo intermediário (`grafo.txt`) que é lido pelo módulo C++.
- A função `create_graph` deste arquivo é chamada pela função `initialize_graph` do `agent_tools.py`

3- Execução dos Algoritmos de Busca (C++):

- Os algoritmos Dijkstra e A* foram implementados em C++ por eficiência.
- Um Makefile compila o código. `agent_tools.py` possui a função `compile_cpp_code` que usa esse Makefile.
- A função `run_search` do `agent_tools.py` roda o executável.
- O executável recebe o grafo (`grafo.txt`) por linha de comando e pela entrada padrão recebe origem, destino e algoritmo, gerando a rota e a distância total como saída em um arquivo (`search_results.txt`).

A comunicação entre os componentes é feita por meio de arquivos e chamadas de sistema (`subprocess.run`), garantindo modularidade e fácil depuração.



3. Prompts testados

Comandos

- 1 - Quais são as universidades disponíveis?
- 2 - Quais são as universidades de BH?
- 3 - Liste as universidades
- 4 - Voce pode me dizer quais sao os pontos disponiveis para pesquisa no grafo?
- 5 - Qual o melhor caminho entre Campus Pampulha da Universidade Federal de Minas Gerais e PUC Minas - Campus Barreiro usando astar?
- 6 - Qual o melhor caminho entre Campus Pampulha da Universidade Federal de Minas Gerais e PUC Minas - Campus Barreiro usando dijkstra?
- 7 - Quero a menor rota do CEFET-MG - Campus II até UNIFENAS usando A*
- 8 - Quero a menor rota do CEFET-MG - Campus II até UNIFENAS usando Dijkstra

9 – Utilizando o algoritmo A estrela, voce pode me dizer o percurso mais eficiente entre Ibmec e Faculdade Universo?

10 - Utilizando o algoritmo dikistra, voce pode me dizer o percurso mais eficiente entre Ibmec e Faculdade Universo?

11 - Me mostre a menor distancia entre MIT e Pontifícia Universidade Católica de Minas Gerais com a*

12 - Me mostre a menor distancia entre Faculdade de Ciências Médicas de Minas Gerais e Harvard com Dijkstra

Resultados

Os resultados são muito grandes. Abaixo temos o exemplo de resultado para os comandos 1, 5 e 11. Os resultados para os outros comandos possuem a mesma estrutura desses, do 1 ao 4 a resposta é a mesma, do 5 ao 10 a estrutura da resposta é a mesma, e 11 e 12 também possuem a mesma estrutura. No README estão todos os resultados.

1-

CEFET-MG - Campus II

CEFET-MG - Campus VI

Campus Pampulha da Universidade Federal de Minas Gerais

Centro Esportivo Universo

Centro Universitário UNA

Centro Universitário UNA

Centro Universitário UniHorizontes

Centro Universitário de Belo Horizonte

Escola Guignard - UEMG

Escola de Arquitetura da UFMG

Escola de Design - UEMG

Escola de Veterinária

Escritório de Representação da UFV

Estácio Polo EAD Santa Inês

Estácio de Sá

Faculdade Anhanguera - Campus Antônio Carlos

Faculdade Jesuíta de Filosofia e Teologia

Faculdade Pitágoras

Faculdade Pitágoras

Faculdade Pitágoras

Faculdade Pitágoras

Faculdade Unimed

Faculdade Universo

Faculdade de Ciências Médicas de Minas Gerais

Fundação João Pinheiro - Campus Pampulha

Ibmec

Instituto de Geociências

PUC Minas - Campus Barreiro

PUC Minas - Campus São Gabriel
Pontifícia Universidade Católica de Minas Gerais
UNA - Centro universitário
UNA Aimorés
UNI-BH campus Lagoinha
UNIFENAS
Universidade Federal de Minas Gerais - campus Saúde
Universidade Fumec
Universidade José do Rosário Vellano
Universidade Newton Paiva
Universidade Prof. Edson Antônio Velano - Unifenas - Câmpus BH, Unidade Jaraguá

5-

Rota encontrada com sucesso!

Saindo do no 351982124 que é o(a) Campus Pampulha da Universidade Federal de Minas Gerais,
vamos para o no 27589563,
depois para o no 28404409,
depois para o no 28404508,
depois para o no 8936627355,
depois para o no 8936627353,
depois para o no 448283431,
depois para o no 28404678,
depois para o no 1830888189,
depois para o no 28293871,
depois para o no 28634937,
depois para o no 448294617,
depois para o no 61851825,
depois para o no 1466399473,
depois para o no 1466344681,
depois para o no 94879231,
depois para o no 94879238,
depois para o no 353574346,
depois para o no 94880300,
depois para o no 1735086877,
depois para o no 94895935,
depois para o no 94896557,
depois para o no 10250636625,
depois para o no 29601600,
depois para o no 8213029691
e por fim chegamos ao no 319531371 que é o(a) PUC Minas - Campus Barreiro.
Com uma distância total de 9942.12 metros.

 Imagem da rota: ../data/route.png

 GIF animado da rota: ../data/route_gif.png

11-

MIT esta fora de alcance.

4. Análise dos Resultados

Durante os testes realizados por meio do prompt conversacional, foi possível observar que o tempo total de resposta percebido pelo usuário é significativamente impactado pela geração da imagem (route.png) e, especialmente, pela geração do GIF (route_gif.png). Essa etapa é realizada após a execução do algoritmo de busca e depende diretamente do tamanho do caminho encontrado: quanto maior a quantidade de nós na rota, maior o tempo necessário para a criação do GIF, o que contribui para a impressão de lentidão no sistema.

No entanto, para realizar uma análise justa da eficiência dos algoritmos de busca — Dijkstra e A* —, consideramos apenas o tempo necessário para a geração do arquivo search_result.txt, que representa a saída bruta da execução do algoritmo. Esse tempo é medido logo após o executável tp1 ser chamado pelo sistema, e antes da geração de qualquer imagem.

Ao comparar o desempenho dos algoritmos com diferentes pares de origem e destino, mantendo os mesmos parâmetros e variando apenas o algoritmo utilizado, foi possível constatar que ambos produzem os resultados de forma bastante rápida. Ainda assim, o algoritmo A* apresentou um desempenho levemente superior em cenários em que os nós de origem e destino estavam mais distantes no grafo. Essa melhoria se deve ao uso de heurística que guia a busca em direção ao objetivo, reduzindo o espaço de busca explorado em comparação ao algoritmo de Dijkstra, que considera todos os caminhos possíveis com base apenas no custo acumulado.

Portanto, embora a diferença de tempo seja pequena, especialmente em grafos de tamanho moderado, a utilização do A* pode trazer benefícios de desempenho práticos em buscas mais longas. Para o usuário final, essa diferença só será percebida se etapas como a geração de imagem e GIF forem otimizadas ou realizadas de forma assíncrona, separando claramente o tempo de busca do tempo de visualização.

5. Configurações para teste

- Sistema Operacional do computador: Windows 11;
- Linguagem de programação implementada: Python e C++;
- Compilador utilizado: G++ da GNU Compiler Collection;
- Processador: i7;
- Quantidade de memória RAM: 16GB;
- Placa de vídeo: RTX2060.